

JavaScript - P1 / 2026-2

Tema

Minha Bucket List de Viagens

Você recebeu a estrutura HTML pronta da aplicação. Seu trabalho será desenvolver **somente o arquivo `index.js`**, utilizando JavaScript para tornar a página funcional.

Não altere os IDs, classes ou a estrutura principal existente no arquivo HTML.

Funcionalidades e pontuação

1. Cadastrar uma viagem — 2,0 pontos

Ao clicar em **Adicionar viagem**, o sistema deverá criar uma nova linha na tabela com os dados informados no formulário.

A linha criada deve possuir a classe:

```
<tr class="viagem-item">
```

Ela deverá conter obrigatoriamente os elementos/classes definidos no arquivo `index.js` inicial.

Consulte o modelo de linha comentado dentro de `#listaViagens` em `aluno/index.html`. Ele mostra a estrutura completa, as classes obrigatórias e os formatos esperados. Mantenha o exemplo comentado: a lista deve começar vazia, e as viagens devem ser criadas pelo seu JavaScript com os dados do formulário.

2. Exibir corretamente os dados — 1,5 ponto

Cada viagem deve mostrar:

- foto;
- país/destino;
- data prevista;
- descrição.

3. Situação da viagem — 1,0 ponto

O checkbox **Já fui para este destino** deverá definir o texto exibido na coluna de status:

- marcado: **Já fui**;
- desmarcado: **Quero ir**.

4. Limpar o formulário — 0,5 ponto

Após cadastrar uma nova viagem:

- os campos devem ficar vazios;
- o checkbox deve voltar a desmarcado.

5. Editar uma viagem — 2,0 pontos

Cada viagem deverá ter um botão **Editar**.

Ao clicar nele:

1. os dados da viagem devem voltar para o formulário;
2. o usuário poderá alterar os valores;
3. ao salvar, a viagem original deverá ser atualizada;
4. não deve ser criada uma nova linha.

6. Excluir uma viagem — 1,5 ponto

Cada viagem deverá ter um botão **Excluir**.

Ao clicar nele, somente aquela viagem deverá ser removida da página.

7. Validar os campos — 1,0 ponto

Não permita cadastrar uma viagem quando qualquer um destes campos estiver vazio:

- país;
- foto;
- data;
- descrição.

Nessa situação, exiba o elemento **#mensagemErro**. (remover a classe d-none)

8. Atualizar a interface — 0,5 ponto

O elemento **#contadorViagens** deve mostrar a quantidade de viagens existentes.

Quando a lista estiver vazia, **#listaVazia** deve aparecer (remover a classe d-none). Quando houver pelo menos uma viagem, deve ficar oculto (adicionar a classe d-none).

Regras

- Desenvolva somente `index.js`.
 - Não altere os IDs fornecidos.
 - Não altere as classes obrigatórias utilizadas para identificar os itens cadastrados.
 - Utilize os conceitos trabalhados em aula: seleção de elementos do DOM, leitura/escrita de valores, eventos, condicionais, laços quando necessário e funções.
 - Não é necessário utilizar arrays, banco de dados, API, `localStorage`, frameworks JavaScript ou bibliotecas externas.
-

Testando a correção

ANTES DE TUTOOOOOOOOOOOOOOOOOOO!

1. Certifique-se que a sua implementação esteja como `index.js` e dentro do diretório aluno.
2. Renomei o diretório aluno para o nome da dupla. Ex: `joao_silva_e_maria_antonia`.

OBS: não utilize espaços nem acentos.

3. Altere o src do iframe (linha 65 do arquivo avaliador.html) de `../aluno/index.html` para `../nome_da_dupla/index.html`.

```
<iframe id="appAluno"
src="../joao_silva_e_maria_antonia/index.html"></iframe>
```

AGORA SIM VOCÊ ESTÁ PRONTO PARA TESTAR, PEGUE UM CAFÉZIM (:

Por segurança do navegador, abra o pacote por um servidor HTTP local.

Opção simples no VS Code:

1. instale a extensão **Live Server**;
2. abra a pasta do pacote;
3. clique com o botão direito em `corretor/avaliador.html`;
4. escolha **Open with Live Server**;
5. clique em **Executar correção**.

O corretor:

- preenche o formulário;
- dispara os eventos da aplicação;
- confere o DOM produzido;
- testa cadastro, status, limpeza, edição, exclusão, validação e contador;
- mostra exatamente o que falhou;
- calcula a nota final automaticamente.